

IOM — Web API Task Plan

1. Bối cảnh project

IOM là hệ thống quản lý thu/chi cá nhân.

Hiện tại project đã có Telegram Bot để người dùng nhắn câu tự nhiên như:

```
ăn sáng 30k  
đổ xăng 50k  
lương tháng này 5tr
```

Hệ thống sẽ parse câu đó thành dữ liệu có cấu trúc rồi lưu vào database.

Nhiệm vụ của bạn trong phase này là làm phần **Web API** để sau này frontend web có thể gọi được.

2. Mục tiêu tổng quát

Làm các REST API cơ bản cho web client:

```
Auth API  
User API  
Transaction API  
Summary API
```

Trong đó ưu tiên cao nhất là:

1. Register
2. Login
3. Get current user
4. CRUD transaction
5. List transaction có filter/pagination
6. Summary thu/chi theo tháng

3. Nguyên tắc làm việc

Project này không viết controller thừa bừa.

Luồng chuẩn nên là:

```
Web Controller
  ↓
Application Use Case / Service
  ↓
Repository
  ↓
Domain Entity
  ↓
Database
```

Không để business logic nằm trong Controller.

Controller chỉ làm các việc:

- Nhận request
- Validate input
- Lấy current user nếu cần
- Gọi service/use case
- Trả response

Service/use case làm các việc:

- Kiểm tra logic nghiệp vụ
- Gọi repository
- Map entity sang response DTO
- Throw exception nếu lỗi

Repository chỉ làm việc với database.

4. Quy ước package đề xuất

Tạo theo structure này để code dễ quản lý:

```
src/main/java/me/nghtlong3004/iom/api/
├── channel/
│   └── web/
│       ├── auth/
│       └── AuthController.java
```

```

|
|
|   ├── dto/
|   |   ├── LoginRequest.java
|   |   ├── RegisterRequest.java
|   |   ├── AuthResponse.java
|   |   └── CurrentUserResponse.java
|
|   ├── user/
|   |   ├── UserController.java
|   |   └── dto/
|   |       ├── UpdateProfileRequest.java
|   |       └── UserProfileResponse.java
|
|   ├── transaction/
|   |   ├── TransactionController.java
|   |   └── dto/
|   |       ├── CreateTransactionRequest.java
|   |       ├── UpdateTransactionRequest.java
|   |       ├── TransactionResponse.java
|   |       └── TransactionListResponse.java
|
|   └── summary/
|       ├── SummaryController.java
|       └── dto/
|           ├── MonthlySummaryResponse.java
|           └── CategorySummaryResponse.java
|
|── application/
|   ├── service/
|   |   ├── AuthService.java
|   |   ├── JwtTokenService.java
|   |   ├── UserService.java
|   |   ├── TransactionService.java
|   |   └── SummaryService.java
|
|   ├── repository/
|   |   ├── AppUserRepository.java
|   |   └── TransactionRepository.java
|
|   └── common/
|       ├── error/
|       |   ├── GlobalExceptionHandler.java
|       |   ├── ErrorCode.java
|       |   └── BusinessException.java
|       └── response/

```

```
|— ApiResponse.java
|— PageResponse.java
```

5. Sprint plan

Sprint 0 — Setup & đọc project

Mục tiêu

Hiểu project đang có gì trước khi code.

Checklist

- ☐ Chạy được project local.
- ☐ Chạy được PostgreSQL bằng Docker Compose.
- ☐ Check được app start không lỗi.
- ☐ Đọc README.
- ☐ Đọc migration SQL:
- ☐ `V1__create_app_users.sql`
- ☐ `V2__create_external_accounts.sql`
- ☐ `V3__create_transactions.sql`
- ☐ Đọc entity:
- ☐ `AppUser`
- ☐ `Transaction`
- ☐ `ExternalAccount`
- ☐ Đọc config:
- ☐ `SecurityConfig`
- ☐ `JwtConfig`
- ☐ `ApplicationConfig`
- ☐ `CustomUserDetailsService`

Output cần có

Gửi lại cho team một note ngắn:

- Project chạy bằng lệnh nào?
- Database có những bảng nào?
- Hiện tại đã có security gì?
- API nào cần làm trước?

Sprint 1 — Auth API

Mục tiêu

Làm được đăng ký, đăng nhập, lấy thông tin user hiện tại.

Task 1.1 — Tạo repository cho user

File cần tạo/sửa

```
repository/AppUserRepository.java
```

Yêu cầu

Repository cần có:

```
Optional<AppUser> findByEmail(String email);  
  
boolean existsByEmail(String email);
```

Done khi

- [] Query được user theo email.
 - [] Check được email đã tồn tại hay chưa.
-

Task 1.2 — Tạo DTO cho Auth

Folder

```
channel/web/auth/dto/
```

File cần tạo

```
RegisterRequest.java  
LoginRequest.java  
AuthResponse.java  
CurrentUserResponse.java
```

RegisterRequest

```
{
  "email": "user@example.com",
  "password": "12345678",
  "firstName": "Nguyen",
  "lastName": "Long"
}
```

Validation:

```
email: required, email format
password: required, min 8 chars
firstName: optional, max 35 chars
lastName: optional, max 20 chars
```

LoginRequest

```
{
  "email": "user@example.com",
  "password": "12345678"
}
```

Validation:

```
email: required, email format
password: required
```

AuthResponse

```
{
  "accessToken": "jwt_token_here",
  "tokenType": "Bearer",
  "expiresIn": 3600,
  "user": {
    "id": 1,
    "email": "user@example.com",
    "firstName": "Nguyen",
    "lastName": "Long",
    "role": "USER"
  }
}
```

```
}  
}
```

Task 1.3 — Tạo JwtTokenService

File cần tạo

```
application/service/JwtTokenService.java
```

Nhiệm vụ

Service này chịu trách nhiệm generate access token.

Token cần có claim:

```
sub = user email  
user_id = user id  
scope = ROLE_USER hoặc ROLE_ADMIN  
iat = issued at  
exp = expired at
```

Config đề xuất

Thêm vào `application-dev.yml` hoặc config tương ứng:

```
iom:  
  security:  
    jwt:  
      access-token-expiration-seconds: 3600
```

Done khi

- [] Login xong trả được JWT.
- [] JWT dùng được với endpoint protected.
- [] Token hết hạn theo config.

Task 1.4 — Tạo AuthService

File cần tạo

```
application/service/AuthService.java
```

Method cần có

```
AuthResponse register(RegisterRequest request);

AuthResponse login(LoginRequest request);

CurrentUserResponse getCurrentUser(Authentication authentication);
```

Logic register

1. Check email đã tồn tại chưa.
2. Nếu tồn tại → throw lỗi EMAIL_ALREADY_EXISTS.
3. Encode password bằng BCrypt.
4. Tạo AppUser với authProvider = LOCAL, role = USER, isActive = true.
5. Save vào database.
6. Generate access token.
7. Trả AuthResponse.

Logic login

1. Dùng AuthenticationManager để authenticate email/password.
2. Nếu sai email/password → trả 401.
3. Nếu user inactive → trả 403.
4. Generate access token.
5. Trả AuthResponse.

Done khi

- [] Register tạo user mới trong DB.
- [] Password trong DB là hash, không lưu plain text.
- [] Login đúng thì trả access token.
- [] Login sai password thì trả 401.
- [] Register trùng email thì trả 409.

Task 1.5 — Tạo AuthController

File cần tạo

```
channel/web/auth/AuthController.java
```

Endpoint cần có

```
POST /api/v1/auth/register  
POST /api/v1/auth/login  
GET /api/v1/auth/me
```

API contract

Register

```
POST /api/v1/auth/register  
Content-Type: application/json
```

Request:

```
{  
  "email": "user@example.com",  
  "password": "12345678",  
  "firstName": "Nguyen",  
  "lastName": "Long"  
}
```

Response `201 Created`:

```
{  
  "success": true,  
  "message": "Register successfully",  
  "data": {  
    "accessToken": "jwt_token_here",  
    "tokenType": "Bearer",  
    "expiresIn": 3600,  
    "user": {  
      "id": 1,  
      "email": "user@example.com",  
      "firstName": "Nguyen",  
      "lastName": "Long",  
    }  
  }  
}
```

```
    "role": "USER"
  }
}
```

Login

```
POST /api/v1/auth/login
Content-Type: application/json
```

Request:

```
{
  "email": "user@example.com",
  "password": "12345678"
}
```

Response 200 OK:

```
{
  "success": true,
  "message": "Login successfully",
  "data": {
    "accessToken": "jwt_token_here",
    "tokenType": "Bearer",
    "expiresIn": 3600,
    "user": {
      "id": 1,
      "email": "user@example.com",
      "firstName": "Nguyen",
      "lastName": "Long",
      "role": "USER"
    }
  }
}
```

Me

```
GET /api/v1/auth/me
Authorization: Bearer <access_token>
```

Response 200 OK:

```
{
  "success": true,
  "message": "Get current user successfully",
  "data": {
    "id": 1,
    "email": "user@example.com",
    "firstName": "Nguyen",
    "lastName": "Long",
    "avatarUrl": null,
    "role": "USER"
  }
}
```

Done khi

- [] Register chạy được bằng Postman/cURL.
- [] Login chạy được bằng Postman/cURL.
- [] Gọi `/me` không token → 401.
- [] Gọi `/me` có token hợp lệ → 200.
- [] Không expose `passwordHash` ra response.

Sprint 2 — User Profile API

Mục tiêu

Web client xem và cập nhật thông tin cá nhân.

Task 2.1 — Get profile

Endpoint

```
GET /api/v1/users/me
Authorization: Bearer <access_token>
```

Response

```
{
  "success": true,
  "message": "Get profile successfully",
  "data": {
```

```
{
  "id": 1,
  "email": "user@example.com",
  "firstName": "Nguyen",
  "lastName": "Long",
  "avatarUrl": null,
  "role": "USER",
  "authProvider": "LOCAL",
  "createdAt": "2026-06-03T10:00:00Z"
}
```

Done khi

- [] User chỉ xem được profile của chính mình.
- [] Không trả password hash.

Task 2.2 — Update profile

Endpoint

```
PATCH /api/v1/users/me
Authorization: Bearer <access_token>
Content-Type: application/json
```

Request

```
{
  "firstName": "Hoang Long",
  "lastName": "Nguyen",
  "avatarUrl": "https://example.com/avatar.png"
}
```

Response

```
{
  "success": true,
  "message": "Update profile successfully",
  "data": {
    "id": 1,
    "email": "user@example.com",
    "firstName": "Hoang Long",
    "lastName": "Nguyen",
  }
}
```

```
"avatarUrl": "https://example.com/avatar.png",  
"role": "USER"  
}  
}
```

Done khi

- [] Update được firstName.
- [] Update được lastName.
- [] Update được avatarUrl.
- [] Không cho update role/email/password qua API này.

Sprint 3 — Transaction CRUD API

Mục tiêu

Web client có thể tạo, xem, sửa, xóa giao dịch thu/chi.

Task 3.1 — Tạo TransactionRepository

File cần tạo/sửa

```
repository/TransactionRepository.java
```

Method cần có

```
Page<Transaction> findById(Long userId, Pageable pageable);  
  
Optional<Transaction> findByIdAndUserId(Long id, Long userId);
```

Có thể bổ sung query filter sau:

```
type  
category  
fromDate  
toDate  
minAmount  
maxAmount  
keyword
```

Task 3.2 — Create transaction

Endpoint

```
POST /api/v1/transactions
Authorization: Bearer <access_token>
Content-Type: application/json
```

Request

```
{
  "type": "EXPENSE",
  "amount": 30000,
  "currency": "VND",
  "category": "FOOD",
  "note": "ăn sáng",
  "occurredAt": "2026-06-03T07:30:00Z"
}
```

Response 201 Created

```
{
  "success": true,
  "message": "Create transaction successfully",
  "data": {
    "id": 1,
    "type": "EXPENSE",
    "amount": 30000,
    "currency": "VND",
    "category": "FOOD",
    "note": "ăn sáng",
    "sourcePlatform": "WEB",
    "occurredAt": "2026-06-03T07:30:00Z",
    "createdAt": "2026-06-03T07:31:00Z"
  }
}
```

Validate

```
type: required, INCOME hoặc EXPENSE
amount: required, > 0
currency: default VND
```

```
category: required
note: optional, max 500 chars
occurredAt: optional, default now
```

Done khi

- [] Tạo transaction gắn đúng current user.
- [] Không cho user truyền userId từ client.
- [] `sourcePlatform` set là `WEB`.
- [] Amount <= 0 thì trả 400.

Task 3.3 — List transactions

Endpoint

```
GET /api/v1/transactions?
page=0&size=20&type=EXPENSE&category=FOOD&from=2026-06-01&to=2026-06-30
Authorization: Bearer <access_token>
```

Query params

```
page: default 0
size: default 20, max 100
type: optional
category: optional
from: optional, yyyy-MM-dd
to: optional, yyyy-MM-dd
sort: optional, default occurredAt,desc
```

Response

```
{
  "success": true,
  "message": "Get transactions successfully",
  "data": {
    "items": [
      {
        "id": 1,
        "type": "EXPENSE",
        "amount": 30000,
        "currency": "VND",
        "category": "FOOD",
```

```
    "note": "ăn sáng",
    "sourcePlatform": "WEB",
    "occurredAt": "2026-06-03T07:30:00Z"
  },
],
"page": 0,
"size": 20,
"totalElements": 1,
"totalPages": 1,
"hasNext": false
}
}
```

Done khi

- [] User chỉ thấy transaction của chính mình.
- [] Có pagination.
- [] Có filter theo type.
- [] Có filter theo category.
- [] Có filter theo date range.
- [] Sort mặc định theo `occurredAt desc`.

Task 3.4 — Get transaction detail

Endpoint

```
GET /api/v1/transactions/{id}
Authorization: Bearer <access_token>
```

Done khi

- [] User chỉ xem được transaction của chính mình.
- [] Nếu không tồn tại → 404.
- [] Nếu transaction thuộc user khác → 404, không trả 403 để tránh lộ dữ liệu.

Task 3.5 — Update transaction

Endpoint

```
PATCH /api/v1/transactions/{id}
Authorization: Bearer <access_token>
Content-Type: application/json
```

Request

```
{
  "amount": 35000,
  "category": "FOOD",
  "note": "ăn sáng + nước",
  "type": "EXPENSE",
  "occurredAt": "2026-06-03T07:30:00Z"
}
```

Done khi

- [] Update partial được.
- [] Field nào null thì giữ nguyên.
- [] Không cho đổi userId.
- [] Không cho sửa transaction của user khác.

Task 3.6 — Delete transaction

Endpoint

```
DELETE /api/v1/transactions/{id}
Authorization: Bearer <access_token>
```

Response

```
{
  "success": true,
  "message": "Delete transaction successfully",
  "data": null
}
```

Done khi

- [] Xóa được transaction của chính mình.
- [] Không xóa được transaction của user khác.
- [] Nếu không tồn tại → 404.

Sprint 4 — Summary API

Mục tiêu

Web dashboard có số liệu tổng quan.

Task 4.1 — Monthly summary

Endpoint

```
GET /api/v1/summaries/monthly?month=2026-06
Authorization: Bearer <access_token>
```

Response

```
{
  "success": true,
  "message": "Get monthly summary successfully",
  "data": {
    "month": "2026-06",
    "totalIncome": 5000000,
    "totalExpense": 1200000,
    "balance": 3800000,
    "transactionCount": 25
  }
}
```

Done khi

- [] Tính tổng income trong tháng.
- [] Tính tổng expense trong tháng.
- [] Tính balance = income - expense.
- [] Chỉ tính dữ liệu của current user.

Task 4.2 — Category summary

Endpoint

```
GET /api/v1/summaries/categories?month=2026-06&type=EXPENSE
Authorization: Bearer <access_token>
```

Response

```
{
  "success": true,
  "message": "Get category summary successfully",
  "data": [
    {
      "category": "FOOD",
      "totalAmount": 700000,
      "transactionCount": 15
    },
    {
      "category": "TRANSPORT",
      "totalAmount": 300000,
      "transactionCount": 6
    }
  ]
}
```

Done khi

- [] Group theo category.
- [] Filter được theo month.
- [] Filter được theo type.
- [] Chỉ tính dữ liệu của current user.

Sprint 5 — Error handling & API response chuẩn

Mục tiêu

API trả lỗi thống nhất, frontend dễ xử lý.

Task 5.1 — ApiResponse wrapper

File cần tạo

```
common/response/ApiResponse.java
```

Format success

```
{
  "success": true,
  "message": "Login successfully",
  "data": {}
}
```

Format error

```
{
  "success": false,
  "message": "Email already exists",
  "errorCode": "EMAIL_ALREADY_EXISTS",
  "details": []
}
```

Task 5.2 — GlobalExceptionHandler

File cần tạo

```
common/error/GlobalExceptionHandler.java
```

Cần handle

```
MethodArgumentNotValidException → 400
BadCredentialsException → 401
AccessDeniedException → 403
BusinessException → tùy status
EntityNotFoundException hoặc custom NotFoundException → 404
Exception → 500
```

Done khi

- ☐ Validate lỗi trả 400.
 - ☐ Login sai trả 401.
 - ☐ Không có token trả 401.
 - ☐ Không có quyền trả 403.
 - ☐ Resource không tồn tại trả 404.
 - ☐ Không leak stacktrace ra response.
-

Sprint 6 — Test & documentation

Mục tiêu

API làm xong phải chứng minh chạy được.

Task 6.1 — Viết test tối thiểu

Ưu tiên test các service quan trọng:

```
AuthServiceTest
TransactionServiceTest
SummaryServiceTest
```

Checklist:

- ☐ Register success.
 - ☐ Register duplicate email.
 - ☐ Login success.
 - ☐ Login wrong password.
 - ☐ Create transaction success.
 - ☐ User A không xem được transaction của User B.
 - ☐ Monthly summary tính đúng.
-

Task 6.2 — Tạo Postman collection hoặc file curl

Tạo file:

```
docs/api/web-api-curl.md
```

Nội dung cần có:

1. Register
2. Login
3. Copy access token
4. Get me
5. Create transaction
6. List transactions
7. Update transaction
8. Delete transaction
9. Monthly summary
10. Category summary

Mỗi API phải có example request và response.

6. Thứ tự ưu tiên làm

Không làm lan man. Làm theo thứ tự này:

- P0 – Bắt buộc
1. Register
 2. Login
 3. Get current user
 4. Create transaction
 5. List transactions
- P1 – Nên có
6. Get transaction detail
 7. Update transaction
 8. Delete transaction
 9. Monthly summary
- P2 – Sau cùng
10. Category summary
 11. Update profile
 12. Refresh token
 13. Logout
 14. Google OAuth web flow
-

7. Branch & PR convention

Mỗi nhóm task tạo một branch riêng.

```
feature/web-auth-api
feature/web-user-api
feature/web-transaction-api
feature/web-summary-api
feature/api-error-handling
```

Commit message nên viết rõ:

```
feat(auth): add register api
feat(auth): add login api
feat(transaction): add create transaction api
fix(auth): handle duplicate email
test(transaction): add transaction service test
docs(api): add curl examples
```

Mỗi PR phải có mô tả:

```
## What changed?

- Added ...
- Updated ...
- Fixed ...

## How to test?

```bash

curl ...
```

### Checklist

- ☐ Code runs locally
- ☐ API tested by Postman/cURL
- ☐ No passwordHash in response
- ☐ Current user data only
- ☐ Error response is clear

---

## # 8. Definition of Done

Một task chỉ được coi là xong khi đủ các điều kiện:

```text

- Code compile không lỗi.
- App start được.
- API chạy được bằng Postman/cURL.
- Có validate input.
- Có xử lý lỗi cơ bản.
- Không expose dữ liệu nhạy cảm.
- User chỉ thao tác được dữ liệu của chính mình.
- Có example request/response trong docs.
- Có ít nhất test service hoặc manual test note.

9. Một số rule quan trọng không được vi phạm

Rule 1 — Không nhận userId từ client

Sai:

```
{  
  "userId": 1,  
  "amount": 30000  
}
```

Đúng:

Lấy user từ JWT token.

Vì client có thể giả mạo userId.

Rule 2 — Không trả passwordHash

Sai:


```
{
  "id": 1,
  "email": "user@example.com",
  "passwordHash": "$2a$10..."
}
```

Đúng:

```
{
  "id": 1,
  "email": "user@example.com"
}
```

Rule 3 — Không để business logic trong Controller

Sai:

Controller tự check email, tự encode password, tự save DB, tự generate token.

Đúng:

Controller gọi AuthService.
AuthService xử lý nghiệp vụ.

Rule 4 — Transaction phải thuộc current user

Mọi API transaction đều phải filter theo current user:

```
findByIdAndUserId(transactionId, currentUserId)
```

Không dùng:

```
findById(transactionId)
```

Vì nếu dùng `findById`, user A có thể đoán id và xem transaction của user B.

Rule 5 — Error phải rõ ràng

Ví dụ:

```
{
  "success": false,
  "message": "Email already exists",
  "errorCode": "EMAIL_ALREADY_EXISTS"
}
```

Không trả lỗi chung chung kiểu:

```
{
  "error": "Something went wrong"
}
```

10. Checklist cuối cùng để báo cáo lại

Sau khi làm xong, gửi lại cho team:

```
## Completed APIs

- [ ] POST /api/v1/auth/register
- [ ] POST /api/v1/auth/login
- [ ] GET /api/v1/auth/me
- [ ] GET /api/v1/users/me
- [ ] PATCH /api/v1/users/me
- [ ] POST /api/v1/transactions
- [ ] GET /api/v1/transactions
- [ ] GET /api/v1/transactions/{id}
- [ ] PATCH /api/v1/transactions/{id}
- [ ] DELETE /api/v1/transactions/{id}
- [ ] GET /api/v1/summaries/monthly
- [ ] GET /api/v1/summaries/categories

## Evidence

- Link PR:
- Screenshot Postman:
- cURL file:
```

- Test result:
- Note lỗi/chưa làm được:

11. Gợi ý scope nếu chỉ có ít thời gian

Nếu chỉ có 1 ngày, làm:

1. Register
2. Login
3. Get me
4. Create transaction
5. List transactions

Nếu có 2 ngày, làm thêm:

6. Update transaction
7. Delete transaction
8. Monthly summary
9. Error handling chuẩn

Nếu có 3 ngày trở lên, làm thêm:

10. Category summary
11. Profile update
12. Test
13. API docs
14. Refresh token/logout